Sirikh: Project Documentation

Oct 16, 2026 · @Vedanth

Overview

Sirikh is a home-screen expense app for V and K that writes every expense straight into their Google Sheet, so nobody opens the sheet to fill in rows. It runs on iPhone as an installed web app, works offline, and keeps budgets, splits and car fuel stats up to date on every save.

Item	Value
Live app	vedsirimalla.github.io/SIRIKH
Code repository	github.com/VedSirimalla/SIRIKH
Spreadsheet	Balance_Tracker. (tabs: Sirikh, Sonata)
Backend	Google Apps Script web app bound to the spreadsheet, `Code.gs`
App version (bottom of the app)	v16
Backend version (the /exec page)	Version 15
Users	V and K, one shared passphrase

What it covers:

- **Joint**: shared day-to-day spending, deducted from a monthly joint budget and split 50/50.
- **Fixed**: rent and bills (Rent, Utilities, Electricity, Wi-Fi, Misc), split 50/50, no budget.
- **Personal**: one person's own spending, against that person's personal budget; owed back in full when it goes on the other person's card.
- **Sonata**: car costs and fuel for the 2026 Sonata Hybrid, in its own tab, with turn-based fuel paying, MPG, cost per mile and charts.

Repository files

All files sit at the top level of the VedSirimalla/SIRIKH repository (no folders, so drag-and-drop uploads work). Everything except `Code.gs` is the frontend served by GitHub Pages; `Code.gs` is a copy kept for reference and is pasted into Apps Script to run.

File	Runs on	What it does
`index.html`	Phone (browser)	The app screen: header, budget cards (joint, personal), Who owes whom cards (joint, personal), Fixed bills card, Sonata dashboard, the entry form, Recent list, and the dialogs (Set monthly budget, Set personal budget, Settings).
`app.js`	Phone	All app logic: form handling per Type, offline queue, sync with the backend, version check, local estimates of budgets and splits, Sonata charts (drawn as SVG), service worker registration and updates. `APP_VERSION` is shown at the bottom (v16).
`styles.css`	Phone	Look and layout: light and dark mode, cards, meters, the Type switch, chips, tables, row colors, charts.
`sw.js`	Phone	Service worker: caches the app files so it opens offline; always asks GitHub for a fresh copy first. Cache name `sirikh-v16` changes with each release.
`manifest.webmanifest`	Phone	Home-screen name, colors, icons, standalone display.
`icon-180.png`, `icon-192.png`, `icon-512.png`	Phone	App icons (teal square, white S, gold $ coin).

File	Runs on	What it does
`Code.gs`	Google Apps Script	The backend. Validates and saves entries, recalculates balances, colors rows, builds every summary table and chart in the sheet, and answers the app. `BACKEND_VERSION` = 15.
`README.md`	GitHub	Short setup and reference guide.

What is never in the repository: the spreadsheet ID, the passphrase (SIRIKH_KEY), the /exec URL, and any expense data. Those live only in Google and on each phone.

Setup and deployment

A release has two halves: the frontend goes to GitHub, the backend goes into Apps Script as a new version of the same deployment. Skipping the second half is the most common cause of problems.

Backend (Apps Script), first time

1. Open the spreadsheet, then **Extensions > Apps Script**.
2. Replace everything in `Code.gs` with the repository's `Code.gs`, then Ctrl+S.
3. **Project Settings > Script properties > Add script property**: `SIRIKH_KEY` = the shared passphrase.
4. **Deploy > New deployment > Web app**: Execute as **Me**, Who has access **Anyone**, then Deploy and approve access (Advanced > Go to Sirikh (unsafe) > Allow).
5. Copy the **Web app URL** ending in `/exec`. Opening it in an incognito window must say "Sirikh backend is running. Version N."

Backend, every later change

1. Paste the new `Code.gs`, Ctrl+S.
2. **Deploy > Manage deployments**, pick the deployment, pencil icon, **Version: New version**, Deploy.
3. Never use "New deployment" for updates: it creates a different URL the phones do not know. The `/dev` test URL only works while signed in, so the app always uses `/exec`.

Frontend (GitHub Pages)

1. In the repository: **Add file > Upload files**, drag in the changed files, **Commit changes**.
2. Pages is set to **Deploy from a branch: main / (root)** (already on; Save greyed out means it is set). A commit triggers a build; the Actions tab shows a green check in 1 to 2 minutes.

Installing on an iPhone

1. Open the live link in Safari, **Share > Add to Home Screen**.
2. Open Sirikh from the home screen, gear icon, paste the `/exec` URL and the passphrase, Save ("Connected").
3. The home-screen app keeps its own storage, separate from Safari, so settings are entered inside the installed app.

Getting updates on the phone: the app checks GitHub on every open and reloads once when a new version takes over. If the bottom of the app still shows an old version, swipe the app away and reopen; as a last resort delete the icon and add it again.

Using the app

The Type switch (Joint | Fixed | Personal | Sonata) drives everything: it changes the form fields and the cards above the form.

Type	Form fields	Cards shown	Budget
Joint	Date, Category (typed), Amount, Card used, Notes	Joint monthly budget, Who owes whom (joint), Fixed bills (if any)	Joint monthly budget
Fixed	Date, Fixed bill buttons (Rent, Utilities, Electricity, Wi-Fi, Misc), Amount, Card used, Notes	Who owes whom (joint), Fixed bills	None
Personal	Date, Category (typed), Amount, Expense for (V or K), Card used, Notes	Personal budgets (V and K), Who owes whom (personal)	One per person

Type	Form fields	Cards shown	Budget
Sonata	Date, Car category buttons, vendor or gas station, fuel fields (Fuel only), Amount, Expense for (V, K or Joint), Card used, Notes	Sonata stats, Trends charts, Spent so far, Spend by category	None

Common fields

- **Date**: calendar picker, today by default; resets to today when the app reopens on a new day.
- **Card used**: V's card or K's card; remembered for next time.
- **Expense for**: whose expense it is. Personal: V or K (remembered). Sonata: V, K or Joint; follows the card unless changed, so normal turns need no tap. A live hint shows any amount owed before saving.
- **Amount**: number keypad; a negative amount is a refund.

Sonata car categories: Fuel, Servicing, Repairs, Mods, Insurance, Registration/Tax, Tolls/Parking, Car wash, Misc.

Fuel fields: Gas station, Odometer (mi), Range left (mi), Gallons, Price per gallon, Fill (Full when the pump clicked, or Partial), and "I missed logging a fill-up before this one". Amount fills in from gallons x price. A live preview shows miles, MPG and cost per mile, and warns about typos (gallons over the 13.2 gal tank, an odometer lower than the last fill) and fills that look partial.

Buttons and dialogs

- **Set monthly budget** (Joint): starts a new joint budget and a new monthly period for the split.
- **Set personal budget** (Personal): a box for V and one for K; an empty box keeps that person's current budget.
- **Gear icon (Settings)**: the `/exec` URL and the passphrase, stored only on that phone.

Recent list: the last 5 entries of the selected area (main or Sonata), newest first, with a colored stripe and tag, the card used, and details (balance after, miles, MPG). The x deletes an entry after a confirm; unsynced entries show an hourglass and "waiting to sync".

Messages: a green banner confirms each save with the key result ("$605 left", "K owes V $20", "52.7 MPG"); amber means saved on the phone and syncing later; red explains an error.

Sirikh sheet tab

The Sirikh tab holds every Joint, Fixed and Personal entry plus budget rows, one row each, in columns A to I; the script builds the summary tables from column J onward on every save.

Data columns (A to I)

Col	Header	Filled by	Notes
A	Date	App	yyyy-mm-dd
B	Category	App	Store or bill name; budget rows say "Balance set" or "V's personal budget"
C	Amount ($)	App	Negative = refund
D	Card Used	App	V or K (Both on joint budget rows)
E	Notes	App	
F	Type	App / script	Joint, Fixed, Personal, Set balance, Personal budget (Add funds only if typed by hand). Blank becomes Joint
G	Balance ($)	Script	Running joint balance, recalculated top to bottom
H	ID	App	Hidden. Unique id; prevents duplicates and lets the app delete the row
I	Expense For	App	V or K on Personal and Personal budget rows; blank = shared

Row colors

Type	Color
Joint, budget rows	No color
Personal for V	Light blue

Type	Color
Personal for K	Darker blue
Fixed on V′s card	Light orange
Fixed on K′s card	Darker orange

Summary tables (column J onward, current budget period)

Rows	Table	Contents
1 to 8	Who owes whom	V paid, K paid, Total; V′s share, K′s share, Equal split; Settle up (joint); Personal on other′s card; Overall (joint + fixed + personal)
10 to 13	Fixed bills	V paid, K paid, Total; Settle up
15 to 20	Personal expenses	K′s and V′s personal spending on own card and on the other′s card (owed in full); Settle up; Overall
22 to 26	Monthly budgets	Budget, Spent, Remaining for Joint, V personal, K personal (with start date)
28 to 33	Colors	Legend of the row colors
35 onward	History	One row per budget period, newest first: Since, Budget, Joint spent, V paid, K paid, V′s share, K′s share, Settle up, Fixed total, Fixed settle, Personal settle, Overall, V personal spent, K personal spent

Editing by hand is safe in A to I: changing an amount, type, card or Expense For recalculates balances, colors and tables automatically. Never type in column J onward; the script overwrites it.

Sonata sheet tab

The Sonata tab is a separate table for the car, independent of the Sirikh tab and its budget; data sits in columns A to R and the script writes stats, chart data and five charts from column T onward. Its first 15 fill-ups (June 21 to October 5, 2026) were copied once from the closed HyundaiSonata2026 tab, which is only read, never edited.

Data columns (A to R)

Col	Header	Filled by	Notes
A	Date	App	
B	Category	App	Fuel, Servicing, Repairs, Mods, Insurance, Registration/Tax, Tolls/Parking, Car wash, Misc
C	Station / Vendor	App	
D	Amount ($)	App	
E	Card Used	App	V or K
F	Odometer	App	Fuel only
G	Miles Driven	Script	This odometer minus the previous fill's
H	Range Left	App	Miles the dash showed before filling
I	Effective Range	Script	Only when the previous fill was Full
J	Gallons	App	Blank allowed if price is entered
K	Price/Gal	App	
L	MPG	Script	On the Full fill that closes a full-to-full stretch
M	Cost/Mile	Script	Same row as MPG
N	Notes	App	
O	ID	App	Hidden
P	Expense For	App	V, K or Joint (blank = card owner's own turn; "Both" from an early version = Joint)
Q	Fill	App	Full or Partial (blank = Full)
R	Missed Before	App	Yes when a fill-up before this one was not logged

Summary and charts (column T onward)

Rows	Block	Contents
1 to 4	Spent so far	V paid, K paid, Total spent; owed from joint or other-card costs

Rows	Block	Contents
5 to 19	Fuel stats	Overall MPG, rolling MPG (last 5 fills), best and worst MPG, average price per gallon, fuel and total cost per mile, average miles per full tank, average effective range, fill-ups / partial / missed, gallons, fuel spend, miles tracked, last odometer, fill-ups by turn (V, K, Joint)
20 to 30	Spend by category	Each category and the total
40 onward	Chart data	Monthly table (Fuel, Other, Total, Miles, MPG); full-tank log (date, MPG, cost per mile); per-fill log (date, rolling MPG, price per gallon)
Column AI	Charts	MPG per full tank, rolling MPG, price per gallon (every fill), fuel cost per mile, car spend per month

To fix history by hand: type **Partial** in column Q, **Yes** in column R, or V / K / Joint in column P; everything recalculates on the edit.

Formula reference

Every number in the app and the sheet comes from the formulas below; the app estimates them instantly (including unsynced entries) and the script recalculates them in the sheet on every save. A budget period runs from one Set balance row to the next; all Joint, Fixed and Personal totals are for the current period.

Joint budget

```
\text{Balance} = \text{Budget} - \sum \text{Joint amounts since the budget was set}
```

```
\text{Per day} = \frac{\text{Balance}}{\text{days left in the month, including today}}
```

The balance column G runs top to bottom: Set balance sets it, Add funds adds, Joint subtracts, Fixed and Personal leave it unchanged. The meter turns amber at 80% used and red below zero.

Who owes whom (joint, and the same for Fixed)

```
\text{Equal split} = \frac{\text{V paid} + \text{K paid}}{2}
```

```
\text{V's share} = \text{V paid} - \text{Equal split} \qquad \text{K's share}
= \text{K paid} - \text{Equal split}
```

A negative share is what that person owes: V's share of -136.49 reads "V owes K $136.49". Paid amounts count by Card Used.

Personal (owed in full)

```
\text{Personal net (K owes V if positive)} = \text{K's expenses on V's card}
- \text{V's expenses on K's card}
```

Expenses on the person's own card owe nothing; they appear only in the Own card column.

Overall settle-up (Sirikh tab)

```
\text{Overall} = \text{V's joint share} + \text{V's fixed share} +
\text{Personal net}
```

Positive means K owes V. Sonata is settled separately.

Personal budgets

```
\text{Remaining}_{V} = \text{V's personal budget} - \sum \text{Personal
amounts for V since that budget}
```

The same for K. A Personal expense always comes out of the budget of the person it is for, whoever's card paid. Each person's budget restarts at their own latest Personal budget row.

Sonata spending and turns

- V paid, K paid, Total spent: all car costs by Card Used.
- Own turn (Expense For matches the card): nothing owed.
- Joint: split 50/50, so the other person owes half to the card owner.
- A turn paid on the other person's card: owed in full.

Fuel math

Fuel rows are taken in odometer order. A full-to-full stretch runs from one Full fill to the next Full fill; the tank is full at both ends, so every gallon added in between (partials included) is exactly what those miles burned.

$$\text{Miles driven} = \text{Odometer}_{\text{this fill}} - \text{Odometer}_{\text{previous fill}}$$

$$\text{Gallons (if blank)} = \frac{\text{Amount}}{\text{Price per gallon}}$$

$$\text{MPG}_{\text{stretch}} = \frac{\text{Odometer}_{\text{closing Full fill}} - \text{Odometer}_{\text{opening Full fill}}}{\sum \text{gallons of every fill after the opening one, up to the closing one}}$$

$$\text{Cost per mile}_{\text{stretch}} = \frac{\sum \text{amounts in the stretch}}{\text{stretch miles}}$$

$$\text{Effective range} = \text{Miles driven} + \text{Range left} \quad \text{(only when the previous fill was Full)}$$

$$\text{Overall MPG} = \frac{\sum \text{stretch miles}}{\sum \text{stretch gallons}} \qquad \text{Rolling MPG} = \frac{\text{miles over the last 5 fills}}{\text{gallons of those 5 fills}}$$

$$\text{Average price per gallon} = \frac{\text{fuel spend}}{\text{gallons}} \qquad \text{Total car cost per mile} = \frac{\text{all car spend after the first odometer reading}}{\text{miles tracked}}$$

Rules that keep the numbers honest:

- A stretch gets no MPG if it contains a missed fill-up (marked, or more than 1.6 times a typical tank) or a fill whose gallons cannot be known; tracking restarts at that fill.
- Partial rows show no MPG of their own; their gallons count in the next Full fill's stretch.
- Average miles per full tank uses stretches with a single fill (Full straight to Full).
- MPG uses the gallons added at the closing fill, not the previous fill, because those gallons refill exactly what the stretch burned. Dividing effective range by gallons would inflate MPG by about 4 to 6 (range left is miles not yet driven on fuel already in the tank).

How the pieces work together

The app never talks to Google Sheets directly: every save is a small POST to the Apps Script `/exec` URL, and every answer carries the whole fresh state back, which the app stores and draws.

API (one endpoint, `doPost`)

Every request is JSON sent as text (no CORS preflight) with the passphrase in `key`.

Action	Body	Script does
state	`{action:'state'}`	Recalculates both tabs and returns everything
add	`{action:'add', entry}`	Validates, appends a Sirikh row, returns state
add (car)	`{action:'add', table:'car', entry}`	Validates, appends a Sonata row, returns state
delete	`{action:'delete', id}` or with `table:'car'`	Deletes the row with that ID, returns state

The reply holds: version, joint balance and period totals, personal budgets, the last 5 Sirikh entries, and a `car` block (spending, owed, turns, fuel stats, stretches, per-fill log, monthly table, open stretch, last 5 car entries). `doGet` only answers "Sirikh backend is running. Version N." for checking deployments.

Inside the script on each request

1. Check the passphrase against the SIRIKH_KEY script property.
2. Take a script lock (one write at a time).
3. Make sure both tabs exist with the right headers (upgrading older layouts in place).
4. Add or delete the row. A repeated ID is ignored, so retries never duplicate.
5. Recalculate: types, running balance and row colors (Sirikh); miles, MPG, cost per mile and effective range (Sonata).
6. Rewrite the summary areas and keep the five Sonata charts in place.
7. Return the state as JSON.

Offline queue and sync (app.js)

- Every save goes into a queue on the phone first, then sync sends the queue in order.

- No signal: the entry stays queued with an hourglass and the banner says it will sync; it sends when the app is online or reopened.
- Server refusals (bad data) drop that entry with a red message so the rest can continue; a wrong key or URL keeps the queue and asks to fix Settings.
- Budgets, splits, personal budgets and the Sonata open stretch are estimated locally from the last state plus queued entries, so cards update the instant you save.

Version handshake

- The app knows which backend version each kind of entry needs: Fixed needs 7, Personal needs 9, Personal budget needs 10, car entries need 15.
- If the backend is older, those entries are held on the phone (not saved wrongly, for example Fixed as Joint) and a red banner explains the redeploy step; they send automatically once the backend is updated.

Updates and caching

- `sw.js` serves the app offline and always fetches fresh files from GitHub first.
- The app checks for a new service worker on every open and reloads once when it takes over; the bottom of the app shows the version (v16).

Hand edits in the sheet

A simple `onEdit` trigger reruns the recalculation when someone edits the data columns of either tab, so balances, colors, MPG and tables stay right. Edits in the summary areas are ignored (they are overwritten anyway).

Troubleshooting and maintenance

Most problems come from an old copy running somewhere: an old app on the phone or an old script behind the `/exec` link. Check both version numbers first.

Symptom	Cause	Fix
Site shows 404	Pages never built or files in a folder	Files at the top level; make any commit; wait for the green check in Actions

Symptom	Cause	Fix
"Could not reach the sheet"	Access not set to Anyone, or a `/dev` URL	Deploy > Manage deployments > edit: Who has access = Anyone; use the `/exec` URL
`/exec` works but the app fails, `/dev` works	The deployment's version is older than the saved code	Manage deployments > edit > New version > Deploy
"Wrong key"	Passphrase mismatch	Re-enter it in Settings exactly as SIRIKH_KEY (case-sensitive)
"Unable to open the file" in a normal browser window	Signed in to several Google accounts	Test in incognito; the app is not affected
Entries stay with an hourglass	Backend older than the entry needs	Paste the latest `Code.gs`, deploy a New version; the `/exec` page must show the expected version
Fixed saved as Joint	Backend older than version 7	Same as above; then change Type to Fixed in column F
New buttons missing on the phone	Old app copy	Swipe the app away and reopen; the bottom should show the latest version; if not, delete and re-add from Safari
A number looks wrong in the sheet	A row typed by hand with the wrong Type, Fill or Expense For	Fix that cell; the tab recalculates on the edit
MPG spike on one fill	A partial fill marked Full	Type Partial in its Fill cell (Sonata column Q)

Making a change safely

1. Edit the files, bump `APP_VERSION` in `app.js` and the cache name in `sw.js` (both the same number).
2. If `Code.gs` changes in a way the app depends on, bump `BACKEND_VERSION` and the matching number in the app's version check.

3. Upload the changed files to GitHub, then paste `Code.gs` and deploy a New version.
4. Open the `/exec` link (version) and the app (bottom version) to confirm both are live.

Script properties in use: SIRIKH_KEY (passphrase), SONATA_IMPORTED (the one-time copy from HyundaiSonata2026 has happened), SONATA_CHARTS (chart layout version; a different value rebuilds the charts).

Change history and decisions

Backend versions are the number on the `/exec` page; app versions are the number at the bottom of the app. All work happened October 14 to 16, 2026.

Backend	App	Change	Decision behind it
15	v16	Fuel engine: Full / Partial fills, missed fill-ups, full-to-full MPG, rolling and overall MPG, derived gallons, 13.2 gal tank checks, five rebuilt charts; summary moved to column T	Partial fills were inflating MPG; stretches stay exact with any mix of fills
14	v15	Personal grid adds own-card spending	Show all personal spending, not only what is owed
13	v14	Personal mode shows its own Who owes whom table	The joint table was confusing under Personal
12	v13	Sonata Expense For: V, K or Joint	3:1 fuel turns owe nothing; a shared trip fill splits 50/50
11	v12	Sonata split removed; Spent so far only	Turn-based paying made a 50/50 split misleading
10	v11	Personal budgets for V and K; budget card follows the Type	Personal spending needed its own limits
9	v10	Card used and Expense for; owed in full	K's ramen on V's card: K owes V the whole amount
8	v9	Sonata tab: car categories, fuel fields, stats, charts; history imported	Car costs tracked apart from monthly spending

Backend	App	Change	Decision behind it
7	v8	Fixed type (orange, no budget); version handshake	Bills are paid regardless; old scripts saved Fixed as Joint
early	early	Joint budget, Who owes whom, sheet tables, history, offline queue, home-screen app	Replace typing into the sheet with a 10-second phone entry

Planned next (not built): a sign-in version for anyone (Google sign-in with the drive.file permission, the app creating each user's sheet, no Apps Script for users to deploy), and an optional dash trip-MPG field as a cross-check.